Unitree Week 21

Presentation by Mario Ortiz

This Week

- Working on the 2-D Reinforcement Learning Policy command (shoulders only) to a 10-D entire arm RL policy.

Running Commands on the Physical R1 (Hanged on Bracket)

- Power R1 On:
- Power Remote On and confirm connection
- L2 + B → Damping
- L2 + ↑ → Locked Standing
- Select + Y → Wave Hand Command

**NOTE: Test did not succeed; The Wave Hand Command did not initiate.

Physical R1 Hardware

- The basic remote manual does not provide sufficient information to utilize solely upper body commands.
- Unitree has a “real R1-specific high-level locomotion API”.
  - In the official Python SDK, the R1 controller has functions such as “Damp”, “Stance”, “Start”, “WaveHand”, etc. and each of these are mapped as a specific FSM end state.

Unitree XR Teloperation Documentation

- **Important Note:** The R1 EDU could have arrived without having the most recently updated firmware. It is important to verify this.
- XR Teleoperation documentation states that there is a “Regular Mode” which is explicitly supports motion-control workflow.
  - ***This command (indicated as “R1 + A) is not indicated on our physical remote control***

XR Teleoperation “Motion” Wiki

- This wiki has more in depth documentation than of the R1 Manual, describing important sequences (such as “Damping → Locked Standing → Regular Mode”) with **current** button mappings (L2 + B → L2 + UP → R1 + X).
-

Recent R1 EDU Updates

- August 3: Change titles “R1 EDU (5 Dof Arm) motion mode, disable R1-A7 motion mode”
  - This github page explains specific handling for the R1 EDU 5-DoF arm, including the rt/arm_sdk path and R1 arm/head/waist commands”
-

Unitree Explore App

Unitree Explore provides tutorial videos for things like

- Remote control binding
- Robot Control via Remote Controller
- Development & Debugging

Documented binding workflow seems to be: (1) Unitree Explore → (2) Device → (3) Data → (4) RemoteControl Binding → (5) Change → (6) Enter Remote-Control Code

Unitree Explore app is also a way to

10-D RL Architecture

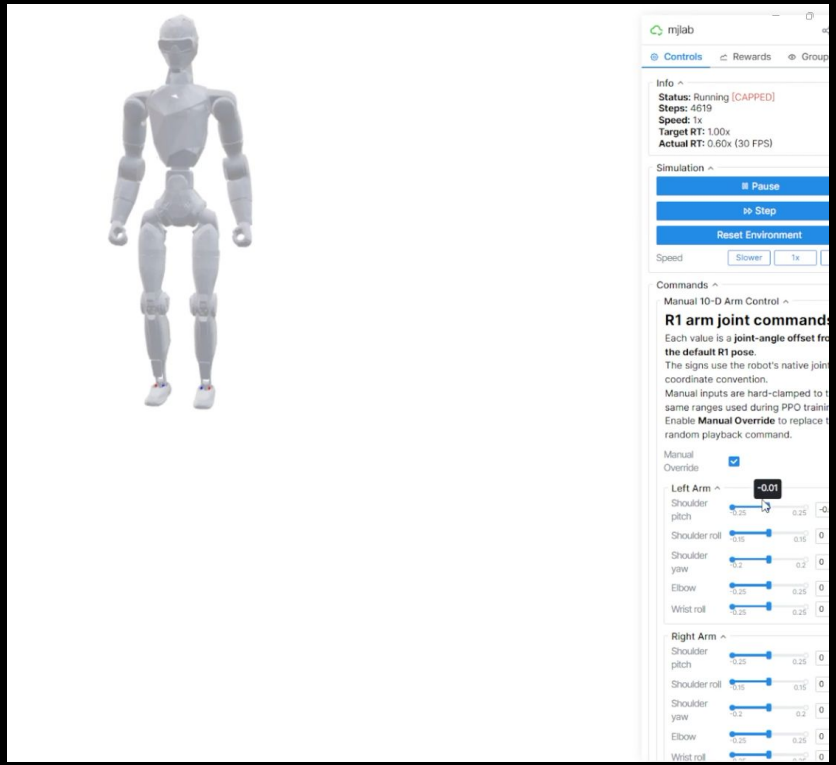
- 82 → PPO → 24
  - 24-D action output
  - 10-D arm command
  - 82 action observations
  - 82 critic observations

Command to run:
(base) mario@LAPTOP-9NUH3CE9:~$ conda activate unitree_rl_mjlab
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~$ cd unitree_rl_mjlab/
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab$ RUN_DIR=$(ls -td logs/rsl_rl/r1_hanger_arms_10d/*smoke_10d_v1* | head -1)

```
CHECKPOINT=$(find "$RUN_DIR" -maxdepth 1 -name 'model_*.pt' \
  -printf '%T@ %p\n' | sort -nr | head -1 | cut -d' ' -f2-)

echo "$CHECKPOINT"
```

- **We can offset the joint angles using the Controls Bar.**



Current RL Design

- The current RL policy involves “joint-space control”; we use the 10 actuated arm joints.
  - $c_{joint} = [q_{L,sp}, q_{L,sr}, q_{L,sy}, q_{L,e}, q_{L,w}, q_{R,sp}, q_{R,sr}, q_{R,sy}, q_{R,e}, q_{R,w}]$
  - What we change are offsets from the R1 default joint angles.
  - The policy rewards the robot when the R1’s joints move to these specific offset joint angles.
-

3D Space RL Policy

- The next step should be instead of commanding where the R1's joints should move to, we will tell the robot where the hands should be in 3D space.
- Reasoning for doing this is that in the 10-D method, we must input joint offset angles for all 10 joints.
  - While in this new method, we can propose the x,y,z coordinate location we want the hands to be located in, and the robot will simply move the necessary joints so that the hands reach this location.

3D Space RL Policy

- New Rewards Needed:
  - Maintaining a near-default posture
  - Joint Limits
  - Avoiding Self-Collision
  - Smooth Motion
  - Low Action Changes
- Command Input Change:
  - Previous: 10 (command) + 24 (joint pos) + 24 (joint velocity)+ 24 (previous action) = 82
  - New: 6 (hand target command) + 6 (current hand positions) + 24 + 24 + 24 = 84
    - This allows the RL policy to comprehend the difference between the current and desired hand positions.

3D Space RL Policy

- Change in rewards:
  - Previous: e_q = q_target - q_actual
  - 6-D: Given p = [x,y,z]:
    - e_L = p_L,target - p_L,actual
    - e_R = p_R,target - p_R,target
    - Reward_hand = exp[(-||e_L||^2 + ||e_R||^2) / σ^2]

Next Steps:

6-D Improvements:

- How do we interface these x,y,z commands in real life?
  - Could be that we set the controller to an “Upper-Body RL Command Mode”, then send x,y,z commands which the robot will move towards (x,y,z commands are respect to initial location of robot).

Sim2Real Integration

- Previously have setup various sim2real integration (DDS communication, coherent LowState, lifecycle handling, stale-data protection, atomic LowCmd, remote input, Sim2Sim Bridge, gantry testing, logging, and physical preflight.
- Now we must ensure that any policy can be transferred into this pipeline (Policy → Sim2Sim → Sim2Real → Physical Robot initiates policy).
-